



南開大學
Nankai University

南 開 大 學

計 算 機 學 院

並行程序設計期末報告

ANNS 並行優化期末報告

姓名：呂昊遠

學號：2410668

專業：計算機科學與技術

指導教師：王剛

2026 年 7 月 12 日

目录

第一部分 优化算法与并行技术	1
一、 引言：ANNS 问题概述	1
二、 ANNS 算法原理与特征	2
(一) 暴力检索 (Flat)	2
(二) 标量量化 (SQ)	2
(三) 乘积量化 (PQ)	3
(四) 倒排文件索引 (IVF)	4
(五) 层级可导航小世界图 (HNSW)	5
(六) 局部敏感哈希 (LSH)	6
三、 并行技术总结	7
(一) SIMD 指令级并行	7
(二) Pthread/OpenMP 多线程并行	8
(三) MPI 多进程并行	10
(四) GPU / CUDA 数据并行	11
(五) 异构协同分流: GPU×CPU 数据并行	11
四、 SimHash-LSH 的跨架构实现	11
(一) CPU 实现	12
(二) GPU 实现	12
(三) 跨架构对比	12
第二部分 综合并行实现	14
五、 算法-架构协同分流	14
(一) 设计动机与思路	14
(二) 融合策略一: 异构协同分流	14
(三) 融合策略二: MPI + 异构分流	15
六、 实验结果与分析	15
(一) 实验环境与策略配置	15
(二) 数据总览	16
(三) LSH 参数权衡	18
(四) 算法与架构的适配与协同	19
(五) 分流占比 f 的选取	20
第三部分 总结与展望	23
七、 并行技术探讨	23
(一) 并行优化规律总结	23
(二) 计算、访存与通信	23

(三) 算法与架构的协同设计	24
(四) 异构融合的适用边界与分布式前景	25
八、 学期总结	26
(一) 实验回顾	26
(二) 展望与心得	26
九、 附录：代码仓库链接	28

第一部分 优化算法与并行技术

一、 引言：ANNS 问题概述

近似最近邻搜索（Approximate Nearest Neighbor Search, ANNS）是一种在高维向量空间中寻找与给定查询向量相近的向量的技术，是当今的信息检索、推荐系统、向量数据库等各种应用的核心算子。其核心描述如下：

给定一个包含 N 条向量的基库 $\mathcal{B} = \{\mathbf{x}_i\}_{i=1}^N \subset \mathbb{R}^d$ ，对于查询向量 $\mathbf{q}$ ，通过 ANNS 在亚线性时间内找到与 $\mathbf{q}$ 最相似的 k 个向量。

对于精确的最近邻搜索（NNS），当数据维度 d 较大或样本量 n 极为庞大时，线性扫描或精确空间划分树等方法的复杂度往往随 d 呈指数增长或随 n 超线性增长，难以满足实时需求。而 ANNS 则通过容忍可控的精度损失，来显著提升查询的速度，同时降低了内存占用和索引构建时间，因而在实际应用中具有不可或缺的重要性。

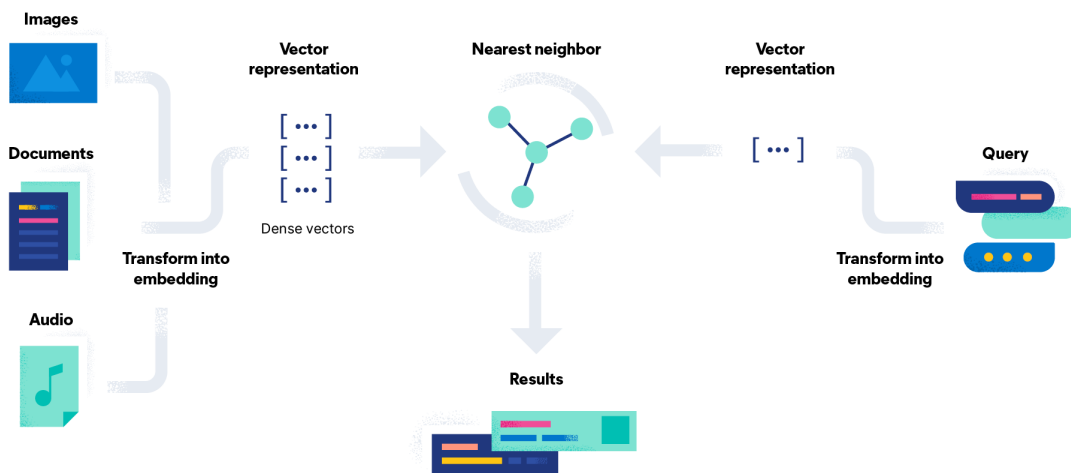


图 1: 工程中的向量检索流程：ANNS 为核心计算模块

为应对精度与效率的权衡，研究者已发展出多种优化的算法，包括基于树的方法（如 KD -树、随机投影树）、基于哈希的方法（如局部敏感哈希 LSH）、基于量化的方法（如乘积量化 PQ 及其倒排索引变体 IVF-PQ）以及基于图的方法（如层次可导航小世界图 HNSW）。不同范式在索引构建时间、查询延迟、召回率和内存消耗等方面各有侧重；衡量性能时通常依靠召回率 $@k$ 、每秒查询数（QPS）和索引大小等指标，并以召回率-延迟权衡曲线直观呈现。随着当今大模型的飞速发展，尽管已有数十年的研究基础，ANNS 仍是一个较为活跃的领域，尤其在动态数据集、分布外查询以及深度神经网络产生的高维嵌入场景中，新的混合策略和基于学习的方法也在不断突破效率与可扩展性的边界。

本学期的实验即围绕这一问题展开。具体实验中，我们统一采用内积（inner product）作为相似度量，距离定义为

$$\text{dis}(\mathbf{q}, \mathbf{x}) = 1 - \langle \mathbf{q}, \mathbf{x} \rangle$$

内积距离在向量归一化后实际等价于余弦距离。评价指标为 $\text{Recall}@k$ ，即返回的 $\text{top-}k$ 结果与真实最近邻的交集比例。

此前的五次实验中，围绕 DEEP100K 数据集 ($N=10^5, d=96$) 系统地探索了 ANNS 在多层次并行架构上的实现与优化：从单核 SIMD 指令级并行 (Lab2)，到多核 Pthread/OpenMP 线程级并行 (Lab3)，再到 MPI 多进程分布式并行 (Lab4)，最后到 GPU/CUDA 数据并行 (Lab5)。本次报告的核心工作在于，将五次实验中分散在各架构、各算法上的工作重新组织并总结，从理论层面梳理各算法和各并行技术层次的原理与特征，并据此设计全新的融合算法与对应的实现，最后系统性地从各个方面探讨不同并行技术的深层规律。

二、 ANNS 算法原理与特征

(一) 暴力检索 (Flat)

暴力检索计算查询 $\mathbf{q}$ 与全部 N 个库向量的精确内积，保留最小的 k 个距离。暴力检索的计算模式极为规整， M 条查询与 N 条基向量的全部内积等价于一个 $M \times N$ 的矩阵乘法，单条查询需 Nd 次乘加与一次对全部 N 个距离的 top- k 选择，计算复杂度为 $O(Nd)$ ，召回率恒为 1。当查询集为 $\mathbf{Q} \in \mathbb{R}^{M \times d}$ 时，全部 $M \times N$ 个内积可表达为一个矩阵乘法：

$$\mathbf{S}_{M \times N} = \mathbf{Q}_{M \times d} \cdot \mathbf{X}_{N \times d}^T \quad (1)$$

其中 $\mathbf{S}[a, i] = \langle \mathbf{q}_a, \mathbf{x}_i \rangle$ 。

从硬件角度来看，它在 GPU 上运算时具有天然的优势，可以通过 cuBLAS 高度优化；而在 CPU 上则效果较差，只能依赖于 cache 进行小幅度的优化。本机测试中，GPU 暴力精确检索的单条延迟为 $25.1 \mu\text{s}$ ，相对 CPU 标量版加速约 62 倍。

暴力检索追求 100% 的召回率，因而在性能上失去了大部分优化的空间，它既是所有近似算法的精度上界，也是衡量其他算法加速比的基线。

(二) 标量量化 (SQ)

标量量化 (Scalar Quantization, SQ) 将每个浮点维度独立量化为低比特整数 (如 int8)，以精度损失换取内存压缩和 SIMD 吞吐，是一种基础而常用的量化算法。本实验采用的是无偏移对称标量量化，全库使用统一的缩放因子，不引入偏移量，以保证量化前后向量内积的排序关系不变。

设基库中所有维度所有元素的最大绝对值为 $v_{\max} = \max_{i,j} |x_{i,j}|$ ，则量化缩放因子为：

$$s = \frac{127}{v_{\max}} \quad (2)$$

对每个浮点元素，直接乘以缩放因子并截断为 int8：

$$\hat{x}_{i,j} = \lfloor x_{i,j} \cdot s \rfloor_{\text{int8}} \in [-127, 127] \quad (3)$$

离线预处理部分需要遍历全库得到 $v_{\max}$ ，并将每个浮点元素量化为 int8。压缩后的向量库大小为原始浮点库的 $1/4$ 。整个计算过程中只需进行一次预处理即可。

进行在线查询时，首先将查询向量 $\mathbf{q}$ 使用同一缩放因子 s 量化为 int8 查询向量 $\hat{\mathbf{q}}$ 。内积计算利用 SIMD 以 16 个 int8 元素为步长展开，单次指令加载 16 个基向量分量与 16 个查询分量，拆分高低 8 位分别做乘法后扩展为 int16 累加至 int32 寄存器 (防止溢出)，一次完成 16 路整型并行乘加：

$$\langle \mathbf{q}, \mathbf{x}_i \rangle \approx \frac{1}{s^2} \sum_{j=1}^d \hat{q}_j \cdot \hat{x}_{i,j} \quad (4)$$

即 int8 内积除以 s^2 反量化还原为浮点近似内积。相比 Flat-SIMD 的 4 路浮点并行, SQ-SIMD 实现了 16 路整型并行, 大幅提升了内积计算吞吐量。在 Lab2 实验中达到了 1.9x 的加速比, 同时还有 0.963 的较高召回率, 若配合轻量 rerank (`rerank_k = 15`), 召回率也可从 0.963 恢复至 0.999 的高精度。

从测试结果来看, 这样的量化方式较好地维护了向量间的相对距离关系, SQ 的精度在很多数据集上已经足够高, 因此重排对其的增益并不明显。SQ 方法也有其他精度更佳的量化方式, 是一类非常有效的加速向量计算的手段, 在当今的大模型量化加速中也有非常广泛的应用。

(三) 乘积量化 (PQ)

乘积量化 (Product Quantization, PQ) 将 d 维向量均匀切分为 M 段, 每段在各自码本 (含 K 个码字) 中查找最近码字, 原向量用 M 个 $\log_2 K$ 比特的码字索引替代。查询时通过距离查找表 (ADC) 累加近似距离, 无需显式重建向量。PQ 可实现数十倍的内存压缩, 是内存受限场景下最优异的压缩检索方案。

(1) 码本训练与向量编码

将向量空间分解为 M 个 d^* 维正交子空间:

$$\mathbf{x} = \left[\underbrace{x_1, \dots, x_{d^*}}_{\mathbf{x}^{(1)}} \mid \dots \mid \underbrace{x_{d-d^*+1}, \dots, x_d}_{\mathbf{x}^{(M)}} \right] \quad (5)$$

对每个子空间 $m \in [M]$, 抽取全部 N 条基向量的第 m 段子向量, 执行 KMeans 聚类 (K 个簇, Lloyd 迭代至收敛), 得到码本 $\mathcal{C}^{(m)} = \{\mathbf{c}_1^{(m)}, \dots, \mathbf{c}_K^{(m)}\}$ 。编码时对每条基向量的每段子向量查找最近质心, 记录其编号 ($0 \sim K-1$):

$$\text{code}(\mathbf{x})_m = \arg \min_{k \in [K]} \|\mathbf{x}^{(m)} - \mathbf{c}_k^{(m)}\|^2, \quad m = 1, \dots, M \quad (6)$$

(2) 非对称距离计算 (ADC) 与 LUT

在进行查询时采用非对称距离计算 (ADC), 查询向量保持浮点精度不量化, 仅使用质心做近似。对于查询 $\mathbf{q}$ 的第 m 段子查询 $\mathbf{q}^{(m)}$, 预计算其与第 m 个子空间 K 个质心的内积, 然后将其存入查找表:

$$\text{LUT}[m][k] = \langle \mathbf{q}^{(m)}, \mathbf{c}_k^{(m)} \rangle, \quad m \in [M], k \in [K] \quad (7)$$

经过上述处理得到 LUT 表之后, 计算基向量 $\mathbf{x}_i$ 的近似内积则只需查表进行累加即可 ($O(M)$ 次 Gather, $M \ll d$):

$$\langle \mathbf{q}, \mathbf{x}_i \rangle_{\text{PQ}} \approx \sum_{m=1}^M \text{LUT}[m][\text{code}(\mathbf{x}_i)_m] \quad (8)$$

(3) 重排 rerank

尽管其优化效果非常显著, 但 PQ 的精度损失是一个不容忽视的问题, 单纯的 PQ 往往面临召回率的崩溃 ($M = 8$ 时仅 0.254), 其原因在于量化噪声使得大量向量的 ADC 距离并列或接近, top- k 选择在概率上几乎退化为随机采样。对此, 我们也在实验中引入了重排 (rerank) 来弥补召回率的损失, 即对于候选集进行原精度的精排。这种两阶段的模式也在后续很多算法中有所体现, 重排阶段带来的额外开销也是需要我们深入考虑与权衡的部分。

第 I 阶段 (粗排) 采用 ADC 扫描全库, 保留 `rerank_k` 个候选 (`rerank_k` 个候选); 第 II 阶段 (精排) 读取这些候选的原始浮点向量, 使用精确的 SIMD 内积重新计算距离并得到最终的 top- k 。在 Lab2 中, $M = 8$, `rerank_k = 500` 使召回率从 0.254 提升至 0.944, 而延迟仅从 1084 μs 增至 1561 μs 。精排阶段仅需处理 `rerank_k` 个候选, 其开销远小于全库扫描。

值得注意的是, PQ 在纯 CPU 环境下的性能瓶颈在于全库 ADC 扫描 ($N \times M$ 次查表和加法, 即 $O(NM)$ 的粗排)。这在 $N = 10^5$ 时并不明显, 但当 N 扩展至上亿级别时, 粗排开销将成为主要的延迟来源, 因而 PQ 也常常与其他方法相结合进行优化 (如 IVF-PQ)。对于单独的 PQ 来说, 其效果则主要体现在空间上的优化, $M = 8, K = 256$ 时算法的空间压缩比可达 48x ($384 \text{ B} \rightarrow 8 \text{ B}$), 这在空间受限的特殊场景下也具有重要的意义。

(四) 倒排文件索引 (IVF)

倒排文件索引 (Inverted File Index, IVF) 采取先分组再搜索的空间划分策略, 以 KMeans 将基向量聚类为 n_{list} 个簇并进行内存重排, 使同簇连续存放以充分利用 cache 的局部性; 查询时先用 query-centroid 距离选出最近的 n_{probe} 个簇 (粗排), 再仅对簇内向量做精确计算 (精排)。IVF 将搜索范围从全库 N 压缩为被探查簇内的向量数 (约 $n_{\text{probe}} \times N/n_{\text{list}}$), 以可控的召回损失换取数量级的计算缩减, 是一种天然的”粗排”策略。

(1) 离线索引构建

索引构建的核心是 KMeans 聚类。设簇数 n_c (即 n_{list} , 默认 256), 聚类的目标为最小化簇内平方和 (WCSS):

$$J(\{\mathbf{c}_j\}, \{a_i\}) = \sum_{i=1}^N \|\mathbf{x}_i - \mathbf{c}_{a_i}\|^2 \quad (9)$$

其中 $\mathbf{c}_j \in \mathbb{R}^d$ 为第 j 个聚类中心, a_i 为 $\mathbf{x}_i$ 的簇分配。Lloyd 迭代交替执行以下两步直至收敛 (实验固定 15 轮):

分配步: 固定中心, 将每条向量分配给 L2 距离最近的中心:

$$a_i^{(t+1)} = \arg \min_{j \in [n_c]} \|\mathbf{x}_i - \mathbf{c}_j^{(t)}\|^2 \quad (10)$$

更新步: 固定分配, 将每个中心更新为其所属向量的均值:

$$\mathbf{c}_j^{(t+1)} = \frac{1}{|\mathcal{S}_j|} \sum_{i \in \mathcal{S}_j} \mathbf{x}_i, \quad \mathcal{S}_j = \{i : a_i^{(t+1)} = j\} \quad (11)$$

初始化采用随机选取训练集中的向量作为初始中心。为提高构建效率, KMeans 只在随机抽取的 $\min(N, 20000)$ 条子集上训练, 聚类完成后再将全库向量分配到最近簇中。

内存重排 (Cache 优化) 是 IVF 索引性能的关键保障。聚类完成后, 基向量按簇归属在内存中连续重排, 同一簇的向量存储在连续的物理区域, 簇边界由 `cluster_starts[0.. $n_c$ ]` 标记 (前 n_c 个元素为起始偏移, 第 n_c 个为总向量数)。同时保存原始 ID 映射表 `reordered_ids` 以在返回结果时还原。查询时对单个簇的扫描则可变为一段连续的线性访问, 以充分发挥 CPU 的硬件预取器和 Cache 的局部性优势。

(2) 两阶段在线查询

IVF 的在线查询也参考了重排的思想, 分为粗排与精排两个阶段, 但这两部分的并行策略具有本质的差异。

第 I 阶段-粗排 (单线程): 计算查询向量与全部 n_c 个聚类中心的内积, 选出距离最近的 n_{probe} 个簇。此阶段计算量较小 ($n_c = 256 \ll N$), 复杂度仅 $O(n_c d)$ 。对于如此小的任务粒度, 多线程的创建与同步开销反而会超过有效计算, 因此粗排阶段保持单线程执行即可。

第 II 阶段-精排 (多线程动态调度): 扫描所选 n_{probe} 个簇内的全部向量, 用 SIMD 内积计算精确距离并维护全局 top- k 。这是 IVF 的主要计算瓶颈。由于各簇大小不均 (受 KMeans 收敛状态影响, 最大簇可能为最小簇的 3-5 倍), 静态分块会导致严重的负载倾斜, 即分配到较大簇的线程成为短板。因此精排阶段采用**簇级动态调度**, 以簇为最小任务单元, 线程从共享任务队列中动态领取下一个未处理的簇, 处理完当前簇后立即领取下一个, 直至全部簇被处理完毕。

Algorithm 1 IVF 检索（粗排单线程 + 精排簇级动态调度）**Input:** 查询 q , 簇心 $\{c_j\}_{j=1}^{n_c}$, 重排库 `reordered_base`, 簇边界 `cluster_starts`, `nprobe`, k **Output:** $\text{top-}k$ 下标

```

1: // 第 I 阶段：粗排（单线程）
2: for  $j = 1$  to  $n_c$  do
3:    $s_j \leftarrow \langle q, c_j \rangle$  ▷ 内积，可用 SIMD
4: end for
5:  $\mathcal{P} \leftarrow \{s_j \text{ 降序排列的 } \text{top-}n\text{probe} \text{ 个簇下标}\}$ 
6: // 第 II 阶段：精排（多线程动态调度）
7: 共享计数器  $\text{next\_idx} \leftarrow 0$ , 各线程局部堆  $H_{\text{local}}[T]$ 
8: #pragma omp parallel for schedule(dynamic, 1) nowait
9: for  $pi = 0$  to  $|\mathcal{P}| - 1$  do
10:    $c \leftarrow \mathcal{P}[pi]$ 
11:   for  $i = \text{cluster\_starts}[c]$  to  $\text{cluster\_starts}[c + 1] - 1$  do
12:      $\text{dist} \leftarrow 1 - \langle q, \text{reordered\_base}[i] \rangle$  ▷ SIMD 内积
13:     更新  $H_{\text{local}}[\text{tid}]$  (容量  $k$ )
14:   end for
15: end for
16: 归并  $H_{\text{local}}[0..T-1]$  为全局  $\text{top-}k$ 
17: return 通过 reordered_ids 映射回原始下标

```

(3) IVF-PQ

IVF 也可以与 PQ 进行结合得到 IVF-PQ 算法，使用 IVF 将候选范围缩减至 `nprobe` 个簇后，在候选簇内用 PQ 的 ADC 查表替代精确 SIMD 内积做粗排，从而进一步降低精排的计算量。本实验实现了两种 IVF-PQ 方案：**全局 PQ** 全库共享同一套 PQ 量化器，LUT 构建仅一次，在所有候选簇间复用；**簇内独立 PQ** 则是每个簇独立训练自己的 PQ 量化器，因各簇内向量分布更集中故量化误差更小，召回率通常更优，但需为每个候选簇分别构建 LUT，额外开销与 `nprobe` 成正比。

在实际测试中，由于 IVF 算法本身已经足够优秀，IVF-PQ 在实验数据集上并未带来明显的收益，IVF 是本学期实验中综合性能最优的 CPU 算法。`nprobe=32` 时，召回率 0.987、延迟 $74.2 \mu s$ ，加速比可达 $21\times$ 。IVF 的关键参数权衡在于 `nlist` 与 `nprobe`，`nlist` 越小簇越大，粗排扫描也就越充分，召回精度更高，但其计算量也会随之增大；反之则相反。

(五) 层级可导航小世界图 (HNSW)

HNSW (Hierarchical Navigable Small World) 以图结构替代空间划分，逐层构建近邻图，查询时从顶层稀疏图向底层稠密图贪心搜索，在度数 M 与构造参数 `efConstruction` 的控制下，可以实现 $O(\log N)$ 的查询复杂度。具体实验时直接使用 `hnswlib` 库构建索引。

HNSW 构建 $L_{\text{max}} + 1$ 层图 ($l = 0$ 为底层，含全部 N 个节点；高层节点数指数衰减)。每个节点在第 l 层最多连接 M 个近邻边 (底层最多 $2M$)。节点所在最高层 $\ell(v)$ 由几何分布抽样确定：

$$\ell(v) = \lfloor -\ln(U(0,1))/\ln M \rfloor, \quad \mathbb{P}(\ell(v) \geq l) = e^{-l \cdot \ln M} \quad (12)$$

进而形成类似跳表的稀疏导航结构。

进行查询时从顶层入口出发，自顶向下逐层贪心导航：在 $l > 0$ 的各层执行 1-贪心搜索（仅找最近点作为下层入口），在底层执行宽度为 ef_search 的集束搜索，同时维护候选集 W 。终止条件为候选集中最近距离已超过结果集中最远距离。其中 ef_search 是权衡精度与延迟的核心参数。

Algorithm 2 HNSW 查询

Input: 查询 q , 顶层入口点 ep , ef , k

Output: top- k 下标

```

1:  $\mathcal{W} \leftarrow \{(ep, \text{dist}(q, x_{ep}))\}$ 
2: for  $l = L_{\max}$  downto 0 do
3:   if  $l > 0$  then
4:      $\mathcal{W} \leftarrow$  层  $l$  上的 1-贪心搜索（仅保留最近点作为下层种子）
5:   else
6:      $\mathcal{W} \leftarrow$  层 0 上的  $ef$ -集束搜索
7:   end if
8: end for
9: return  $\mathcal{W}$  中距离最小的  $k$  个节点
  
```

HNSW 在大规模数据集上优势显著，但其图结构的内存占用量通常高于量化方案，且动态插入/删除的实现复杂度更高。在本实验的数据范围内，该类方法的优势并不明显，需要更大量级的数据集才能体现其高效之处。

(六) 局部敏感哈希 (LSH)

LSH 属于 ANNS 的另一大范式哈希方法，此前学期实验中未涉及，是本报告的新增内容之一。其核心思想是利用随机投影将向量映射到汉明空间，使用汉明距离进行粗排，选出 rerank_k 个候选后再用精确内积进行精排来得到最终结果。

(1) SimHash

本实验实现的是基于随机投影的 SimHash。设 $\mathcal{H} = \{h_{\mathbf{w}} : \mathbb{R}^d \rightarrow \{0, 1\}\}$ 为一族哈希函数，每个函数对应一个标准高斯随机方向 $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$:

$$h_{\mathbf{w}}(\mathbf{x}) = \mathbb{I}[\langle \mathbf{w}, \mathbf{x} \rangle \geq 0] \quad (13)$$

生成 K 个独立随机超平面，每个向量被映射为一个 K -bit 二值码:

$$\text{code}(\mathbf{x}) = (h_{\mathbf{w}_1}(\mathbf{x}), \dots, h_{\mathbf{w}_K}(\mathbf{x})) \in \{0, 1\}^K \quad (14)$$

对于归一化向量 $\mathbf{u}, \mathbf{v}$ ，其夹角为 $\theta = \arccos(\langle \mathbf{u}, \mathbf{v} \rangle)$ ，于是有:

$$\mathbb{P}[h_{\mathbf{w}}(\mathbf{u}) \neq h_{\mathbf{w}}(\mathbf{v})] = \frac{\theta}{\pi}, \quad \mathbb{E}[\text{Hamming}(\text{code}(\mathbf{u}), \text{code}(\mathbf{v}))] = K \cdot \frac{\theta}{\pi} \quad (15)$$

即汉明距离是角距离的无偏估计，可用作 ANNS 粗排的廉价度量方式。两个 K -bit 码的汉明距离仅需一次 XOR 加一次 popcount ($\text{Hamming}(a, b) = \text{popcount}(a \oplus b)$) 即可完成计算，可以得到硬件的指令级支持以得到效率的提升。

(2) 两阶段流程

LSH 在检索时也遵循“粗排—精排”的流程：建库时生成高斯随机矩阵 $\mathbf{W} \in \mathbb{R}^{K \times d}$ ，对所有基向量计算投影 $\mathbf{X} \cdot \mathbf{W}^T$ 并打包符号位为 K -bit 码。查询时则将查询同样投影打包，以 XOR+popcount 计算汉明距离粗排全库，选出 rerank_k 个候选后再用精确内积精排得到最终的 top- k 。

Algorithm 3 SimHash-LSH 建库与查询**Input:** 基库 $\{\mathbf{x}_i\}_{i=1}^N$, 查询 $\mathbf{q}$, K , rerank_k , k **Output:** top- k 下标

```

1: // 建库
2: 生成  $K \times d$  高斯随机矩阵  $\mathbf{W}$ 
3: for  $i = 1$  to  $N$  do
4:    $\text{code}[i] \leftarrow (\mathbb{I}[\langle \mathbf{w}_j, \mathbf{x}_i \rangle \geq 0])_{j=1}^K$  ▷ 符号打包为  $K$ -bit 整数
5: end for
6: // 查询
7:  $\text{qcode} \leftarrow$  查询投影并打包
8: 汉明粗排:  $\forall i$ ,  $\text{Hamming}(\text{qcode}, \text{code}[i])$ , 保留  $\text{rerank\_k}$  个候选
9: 精确精排: 对候选做精确内积, 取 top- $k$ 
10: return top- $k$  下标

```

(3) 跨架构差异

从上文的理论描述到实际的测试结果, LSH 均呈现出极为鲜明的跨架构差异。CPU 上, $K=256$ 、 $\text{rerank_k}=1000$ 时召回率 0.992, 但延迟高达 $1116 \mu\text{s}/q$, 甚至慢于暴力基线 ($0.7\times$), 这是因为汉明粗排仍需扫描全库 $O(N)$, 依靠 CPU 很难在 10 万的量级上发挥 popcount 速度。而 GPU 上, 随机投影则恰好等价于矩阵乘法 ($\text{proj} = \mathbf{X} \cdot \mathbf{W}^\top$), 可以利用 cuBLAS 大幅加速, $K=256$ 、 $\text{rerank_k}=500$ 时延迟仅 $8.7 \mu\text{s}/q$, 加速比可达 $179\times$ 。同一算法在 CPU 和 GPU 上相差如此之大, 显著的性能差异也是架构与算法由其自身特质而相互影响的结果。

三、 并行技术总结

本学期的实验覆盖了从指令级到分布式集群的完整并行层次。本节将每个层次的核心原理与技术特征进行统一总结。

(一) SIMD 指令级并行

SIMD (单指令多数据) 并行利用向量寄存器 (x86 AVX2 为 256-bit, ARM NEON 为 128-bit), 在一个时钟周期内同时对 4~8 个单精度浮点数执行相同运算。在本学期的实验中, SIMD 被用作所有策略的默认基底, 内积计算、L2 距离、PQ 的 LUT 构建等均使用 SIMD 实现指令级的多路并行。

1. float32 内积计算

在本机实验中, Flat 暴力检索与 IVF/IVF-PQ 精排阶段的内积计算均使用 AVX2 的 fused multiply-add (FMA) 指令实现 8 路并行。以 $\langle \mathbf{q}, \mathbf{x} \rangle = \sum_{j=1}^d q_j x_{i,j}$ 为例, 每次加载 $\mathbf{q}$ 和 $\mathbf{x}_i$ 的各 8 个连续分量到 256-bit 寄存器, 执行 `_mm256_fmadd_ps` 完成乘加, 最后对寄存器内 8 个部分和做横向归约 (horizontal sum) 得到最终内积。横向归约需要额外指令 (`_mm256_hadd_ps` 等), 因此向量维度 d 应尽量为 8 的整数倍以避免尾随标量的拖累。在本机的测试结果中, AVX2 使暴力内积从标量的 $4632 \mu\text{s}/q$ 降至 $1562 \mu\text{s}/q$, 考虑纯 SIMD 层面的加速大约为 3 倍。

2. int8 量化的吞吐提升

使用 SQ 方法将 float32 量化为 int8 后, SIMD 吞吐进一步提升。以 ARM NEON 为例 (Lab2 ARM 平台), 量化后单次指令加载 16 个 int8 基向量分量与 16 个 int8 查询分量, 拆分高低 8 位分别做乘法 (vmull_s8, 结果扩展为 int16), 再累加至 int32 寄存器 (vpadal_s16) 防止溢出。相比 float32 的 4 路并行, int8 实现了 16 路并行, 是 SQ 在保持召回率的同时将延迟从 5404 μ s 降至 2796 μ s 的直接原因。

3. SIMD 与内存布局

进行 SIMD 并行优化时, 其加速效率高度依赖内存布局。以 PQ 编码表为例, 朴素方案按向量排列编码, 即 AoS 布局 ($\text{codes}[i][m] = \text{第}i\text{条向量的第}m\text{段编码}$), 在扫描时需要在 M 个子空间之间跳转访存做 gather 操作, 访存地址不连续, 既无法利用 SIMD 的连续加载指令, 也不利于缓存预取。将编码重构为 SoA 布局后, 第 m 个子空间的所有 N 个编码连续存储 ($\text{codes_soa}[m \times N + i]$), 在粗排扫描时变为完全的线性连续访问, CPU 硬件预取器可以提前将后续编码加载到 cache 中。Lab2 的 perf 实测中, SoA 重构使缓存缺失率从 AoS 的 0.582% 降至 0.168%, 体现出内存布局优化带来的收益。

类似地, 在 PQ 部分, 跨质心 SoA 并行 LUT 构建也是 SoA 布局的另一典型应用。传统 PQ 的 LUT 构建需对 $M \times K$ 个质心逐一做内积。将质心也做 SoA 转换后 ($\text{cents_soa}[m \times \text{sub} \times K + d \times K + c]$), 第 m 个子空间同一维度下的连续 K 个质心值被紧密排列。在 AVX2 上, 对于子空间的每个维度 d , 每次使用 `_mm256_set1_ps` 广播查询的单个维度值, 与 SoA 布局下 `_mm256_loadu_ps` 加载的 8 个连续排列的质心对应的维度值做 `_mm256_fmadd_ps`, 从而一次完成 8 路并行的部分积累加。通过上述优化, LUT 构建的复杂度可以从 $O(MKd^*)$ 降至的 $O(MKd^*/8)$, 虽然是离线部分的优化, 但这也能体现出内存布局与 SIMD 相适配时带来的显著收益。

4. popcount 硬件指令

LSH 的汉明距离计算则是 SIMD 的另一个典型场景。两个 K -bit 二值码的汉明距离由 $\text{popcount}(a \oplus b)$ 给出, 只需一次 XOR 加一次 popcount。x86 提供 POPCNT 指令 (单周期处理 64-bit 字), CUDA 提供 `__popc11()` 设备函数 (同样映射为硬件指令)。对于 $K = 256$ 的情况, 每个二值码占 4 个 64-bit 字, 单次汉明距离仅需 4 次 XOR + 4 次 popcount + 3 次加法, 约 0.02 条指令/向量。然而, CPU 上粗排仍需逐向量加载 4 个 64-bit 字, 全库扫描 $N = 10^5$ 时约需 4×10^5 次内存访问——在此瓶颈下 popcount 的计算速度已被内存带宽所覆盖。GPU 上则通过合并访存 (coalesced memory access) 将 $B \times N$ 个汉明距离的计算映射为单一 kernel, 从而与 popcount 的硬件指令吞吐相匹配。这也是 LSH 在 GPU 上远优于 CPU 的原因之一。

(二) Pthread/OpenMP 多线程并行

多线程并行化主要分为两类:

OpenMP 是一种基于编译器指令 (pragma) 的共享内存并行编程模型。其核心优势在于其内置了线程池, 线程仅需创建一次, 后续每次进入 `#pragma omp parallel` 区域时直接唤醒, 避免了重复创建与销毁的开销。其调度策略也由运行时库自动管理, 实际使用中简单便捷, 适合对于规则的数据块进行并行化。

POSIX 线程 (Pthread) 则提供了更底层的线程控制接口。与 OpenMP 相比, Pthread 需要手动管理线程的创建、同步 (mutex / barrier / condition variable) 和销毁, 编码难度较大。其

优势主要在于自身的灵活性, 在实际编程中, 可以根据实际问题来精确控制调度策略与同步粒度等。

1. Pthread 与线程池

对于 Pthread 实现的多线程编程, 需要手动管理复杂的线程调度, 若每次查询都执行 `pthread_create/pthread_join` 循环, 以 2000 次查询 $\times$ 4 线程的典型场景为例, 累计 8000 次线程创建销毁的总开销可达数百毫秒, 完全掩盖了并行计算带来的收益, 最终导致 Pthread 版本出现多线程反而更慢的严重负优化。

其解决方案是使用线程池进行优化, 在查询循环外一次性创建所有的工作线程, 使其在条件变量 (`pthread_cond_t`) 上休眠; 每次查询时通过 `pool.dispatch(fn)` 广播唤醒所需的工作线程, 对其分发任务再等待全部完成。引入线程池后, Pthread 版本的延迟与 OpenMP 趋于一致, 这一思路实际上等价于 OpenMP 内部的线程池机制。

此外, Pthread 的动态调度也同样需要手动编写共享计数器 + 互斥锁来实现。当线程数增至 $T = 8$ 时, 8 个线程频繁争抢同一把互斥锁, 锁竞争的开销与单簇扫描时间 (约 5–10 μs) 处于同一数量级, 这又会导致 Pthread 在 $T = 8$ 时出现扩展停滞甚至回退 (如 PQ Pthread $T=8$ 延迟 738 μs 反高于 $T=4$ 的 668 μs)。从实验结果来看, Pthread 优化效果高度依赖于手动编码, 逻辑上需要有更加细节而深入的理解才能更好地发挥其灵活性而带来持续的收益。对于细粒度的任务调度, 考虑基于原子操作 (如 `std::atomic` 的 `fetch_add`) 或运行时库内置机制 (如 OpenMP 的 `schedule(dynamic)`) 通常优于手动互斥锁方案, 且在编码时更加简便。

2. 跨 query 并行与簇级动态调度

针对 ANNS 问题, 多线程并行主要有两种模式:

跨 query 并行是最简单的思路, 直接将 M 条查询按线程数均匀分块, 各线程独立完成分配到的查询, 线程之间无需同步。这种并行模式适用于 Flat、PQ 这类每条查询的搜索范围固定且计算量均等的算法, 此时使用静态分块 (`schedule(static)`) 即可实现接近线性的加速比。

簇级动态调度是对于 IVF 方法特有的并行策略。IVF 精排阶段需扫描 n_{probe} 个簇, 而在这过程中各簇大小因 KMeans 收敛状态而有所不同, 最大簇可达最小簇的 3–5 倍。若采用静态分块, 分配到大簇的线程将成为短板, 其余线程提前完成后空闲等待, 负载倾斜会带来较大的性能损失。此时使用簇级动态调度则可以很好地解决该问题, OpenMP 使用 `#pragma omp for schedule(dynamic, 1) nowait` 实现, 以单个簇为最小任务单元, 线程完成当前簇后立即从共享任务队列领取下一个, 直至全部簇被处理完毕。`nowait` 子句可以让先完成的线程直接进入归并阶段, 从而无需在隐式 `barrier` 处空转。

3. 线程内局部无锁堆

传统做法中, 多线程共享一个全局 top- k 优先队列, 每次 `push/pop` 都需要加锁保护。当 $k = 10$ 、每条查询扫描数千向量时, 锁竞争频率极高, 多线程加速比随线程数提升而严重退化。针对该问题, 考虑使用线程内局部无锁堆来解决, 即在每个线程内独立维护私有的粗排队列 (容量 `rerank_k`) 和精排队列 (容量 k), 处理所分配的数据块时完全无需任何原子操作或互斥锁; 所有线程完成后, 仅对各线程的精排结果做一次小规模的归并 (合并 $T \times k$ 个元素取 top- k), 以此来消除锁竞争带来的开销。

在 Lab2/PQ-Final 中, 局部无锁堆不仅消除了跨线程同步开销, 还能在统计特性上带来增益。多线程下每个线程从 N/T 的子集中各取 `rerank_k` 个候选, 实际等效候选池规模为 $T \times \text{rerank_k}$,

且其分布更加均匀，各线程从不同数据分块中提取候选，有效避免了局部噪声集中导致的队列阻塞问题。在 $M = 8, \text{rerank_k} = 250$ 配置下，可以达到 0.995 的高召回率与 $1296 \mu\text{s}$ 的延迟，相对 Flat 加速 14.7 倍，在精度与速度上都带来了显著的提升。

4. HNSW 多子图并行

HNSW 的单次贪心搜索具有强串行依赖路径（每步的移动方向依赖上一步的结果，难以在路径内并行），直接进行并行化具有一定的困难。实验时采用的方案是**多子图并行**，将基向量均匀划分为 T 个子集，分别构建 T 个独立子图索引；查询时 T 个线程各自在其负责的子图上独立执行 HNSW 贪心搜索，最后归并各线程的局部 top- k 。该策略用广度换取深度，从而绕开了单图搜索的串行瓶颈，虽然单个子图搜索路径较短，但 T 个线程联合覆盖了更广阔的空间区域，用较小 ef 的多样性弥补了单图深度的不足，也是多线程编程的另一种并行思路。

5. 多线程加速比的边界

根据 Amdahl 定律，若程序中串行部分占比为 α ，则 p 线程的理论加速上限为 $S_p \leq 1/(\alpha + (1 - \alpha)/p)$ 。实验中多线程扩展性的制约因素主要有如下几点：

1. **物理核数限制**：当线程数超过物理核后，多个线程将被调度到同一核上分时执行，加速比往往不再增长甚至可能会回退。
2. **内存带宽瓶颈**：对于访存密集型算法（如 IVF-PQ 的 ADC 查表、Flat 的全库扫描），当线程数增至 4 以上时，多线程同时访问 LUT 与编码表使得 L2/L3 cache 和内存带宽趋于饱和，加速比显著偏离线性。Lab3 实测中，IVF-PQ M1 OMP 从 $T=1$ 的 $437 \mu\text{s}$ 到 $T=8$ 仅降至 $227 \mu\text{s}$ ，算法内加速比仅 $1.93\times$ ，受内存带宽制约较为明显。
3. **归并开销**：每次查询结束需归并 T 个线程的局部 top- k 结果 ($O(Tk \log k)$)，这部分是固定的串行开销，且会随线程数 T 而提升。在低延迟场景下（如 IVF $T=8$ 时总延迟仅 $109 \mu\text{s}$ ），归并阶段固定开销的占比也是不可忽略的一部分。

(三) MPI 多进程并行

MPI 将数据或任务分布到多个进程上，各进程拥有独立的地址空间，通过消息传递 (MPI_Send/Recv、MPI_Gatherv 等) 进行通信。在本学期实验中，MPI 的 ANNS 实现方式包括**数据并行**（将库向量均匀划分给各进程、各进程在自己的子库上独立检索后归并）和**任务并行**（将查询集分派给各进程）。与 OpenMP 共享内存模式不同，MPI 的进程间数据隔离使得它天然适合跨节点分布式场景，但代价是需要显式管理数据划分与结果归并，单机 MPI 已然受通信制约导致扩展性下降，在分布式多节点的网络环境下更是最为主要的性能瓶颈。

MPI_Gatherv 的归并时间随进程数和结果规模 ($M \times k$) 增长。对于本实验中 $M = 2000$ ， $k = 10$ 的场景，进程数的提升是通信开销增大的主要原因，规模上实验的结果集仅约 80 KB，归并开销几乎可以忽略 ($< 0.1 \text{ ms}$)。但在大规模场景下（百万级查询），通信开销也将随之迅速上升。

在 Lab4 中已经探索了 MPI $\times$ OpenMP 混合模式，每个 MPI 进程内部再以 OpenMP 做多线程并行，即 P（进程数） $\times$ T（每进程线程数）的两级并行。该模式在纯 CPU 场景下可以有效利用多核 + 多节点的硬件层次结构。

利用同样的思想, 我们还可以将 GPU 进程也引入 MPI 体系。具体实现时, GPU 进程以 cuBLAS/CUDA kernel 替代内部 OpenMP 线程, 实现 $\text{MPI} \times (\text{GPU} \parallel \text{CPU})$ 的异构混合。这使得 MPI 框架不再局限于同构 CPU 节点, 而是可以将 GPU 作为特殊的算力节点纳入统一的多进程并行, 且在多 GPU 资源的大型工业场景也有一定的参考意义。

(四) GPU / CUDA 数据并行

GPU 拥有数千个轻量级计算核心, 采用 SIMT (单指令多线程) 执行模型, 以海量线程隐藏访存延迟。基于 cuBLAS 库的 GEMM 将 ANNS 最为核心的批量内积计算映射为高度优化的矩阵乘法 ($S = Q_{M \times d} \cdot X_{N \times d}^T$), 可以充分利用 Tensor Core 等硬件加速单元。

1. GPU 优化原则

(1) **大块规整矩阵乘, 减少 kernel 启动频次。**将查询按 batch (本实验默认 $B = 512$) 组织为批处理, 一次 kernel 处理一批查询, 分摊 kernel 启动开销。具体选取时需要注意满足 GPU 的占用率要求, 若 batch 太小则 SM 闲置, 若 batch 太大则可能显存不足。

(2) **设备端归约, 最小化 Host-Device 传输。**全库扫描的全量距离矩阵 ($B \times N \times 4$ 字节) 永远留在设备端, kernel 内部完成 top-k 后只回传 $B \times k$ 个 uint32 下标。 $M = 2000$ 时总传输量约 80 KB, 远低于 800 MB 的原始距离矩阵。D2H (Device-to-Host) 带宽通常为 12–16 GB/s (PCIe 4.0), 传输 800 MB 约需 50–70 ms, 相当于 2.5 万次查询的 GPU 计算时间。因此, 减少 H-D 搬运也是 GPU 优化的重点之一。

(3) **合并访存 (coalesced memory access)。**GPU 的全局内存以 32/128 字节的事务 (transaction) 为单位访问。当 warp 内 32 个线程访问的地址连续且对齐时, 一次事务即可服务整个 warp; 若地址分散, 则需要多次事务甚至逐线程访问。LSH GPU 实现中, 汉明距离 kernel 将线程映射为 $B \times N$ 的 2D 网格, 相邻线程访问相邻的码字地址, 使全局内存事务充分合并。

2. LSH 的 GPU 实现

GPU 上 LSH 查询 kernel 的核心难点在于候选集的选取, CPU 上用堆维护全局 top-rerank_k, 只需要 $O(N \log \text{rerank_k})$, 但 GPU 上每个线程块独立维护堆会导致共享内存膨胀与线程发散。本实验采用直方图定半径法来解决这一问题, 线程协同构建每条查询的汉明距离直方图 (共享内存中 $K + 1$ 个桶), 单线程遍历直方图累积求和确定半径 r 使 $\sum_{h=0}^r \text{hist}[h] \geq \text{rerank_k}$, 再以 r 为阈值收集候选。该方法避免了全局 top-k 的 $O(\text{rerank_k} \cdot N)$ 复杂度, 将其优化为 $O(N + K + C)$ (C 为收集的候选数), 是 GPU LSH 相比 CPU 获得 179× 加速的关键原因之一。

(五) 异构协同分流: GPU×CPU 数据并行

上文中四个层次的并行优化 (SIMD→OpenMP→MPI→GPU) 均为学期实验的已有内容。本次实验在此基础上实现了一种全新的**异构数据并行**, 该方法将查询集按算力比例分派给 GPU 与 CPU, 两块硬件同时运行各自最适合的算法, 并发处理不相交的查询段, 充分发挥不同架构各自的优势, 使最终的吞吐相叠加。

四、SimHash-LSH 的跨架构实现

LSH 算法已在前文介绍了其核心原理 (符号随机投影 → 汉明粗排 → 精排)。接下来将对该算法在 CPU (AVX2+OpenMP) 和 GPU (cuBLAS+CUDA kernel) 两种架构上的完整实现进

行详细讲解。

(一) CPU 实现

CPU 版的核心瓶颈在于投影与汉明扫描均为 $O(NK)/O(N)$ 的全库操作。投影阶段使用 AVX2 的 8 路 FMA 加速, 每条向量的 K 个投影值串行计算; 汉明粗排时以 OpenMP 并行, 各线程维护私有堆、无锁竞争。精排阶段仅对 `rerank_k` 个候选做精确 AVX2 内积。

Algorithm 4 LSH - CPU: 建库与查询

Input: $\text{base} \in \mathbb{R}^{N \times d}$, $\text{query} \in \mathbb{R}^{M \times d}$, K , `rerank_k`, k

Output: 每条查询的 top- k

- 1: **建库:** 随机生成 K 个高斯超平面 $\mathbf{W} \in \mathbb{R}^{K \times d}$
 - 2: **for** $i = 1$ to N **do** ▷ #pragma omp parallel for
 - 3: $\forall j$: $\text{proj}[j] \leftarrow \langle \mathbf{W}[j], \mathbf{x}_i \rangle$ ▷ AVX2 内积
 - 4: 符号位打包为 K -bit 码 $\text{code}[i]$
 - 5: **end for**
 - 6: **查询:** 对 $\mathbf{q}$ 计算 qcode
 - 7: 汉明粗排: #pragma omp parallel, 线程内局部堆保留 $\text{Ham}(\text{qcode}, \text{code}[i])$ 最小的 `rerank_k` 个候选
 - 8: 各线程堆归并为全局 `rerank_k` 候选
 - 9: 精排: 对候选做精确内积 (AVX2), 取 top- k
-

(二) GPU 实现

GPU 版的核心优势在于矩阵运算, 查询投影为 $\text{proj}_Q = \mathbf{Q}_{B \times d} \cdot \mathbf{W}_{K \times d}^\top$ ($B \times K$), 建库投影同理。符号打包、汉明计算、候选选择均以自定义 CUDA kernel 完成。其中汉明距离是 $[0, K]$ 的小整数, 采用**直方图定半径法**代替迭代式 top- k : 线程协同计算每条查询的汉明距离直方图, 单线程遍历桶计数、累积求和直至达到 `rerank_k`, 以此时的汉明半径 r 为阈值, 收集所有满足 $\text{Ham} \leq r$ 的候选。该方法避免了迭代式 top- k 的 $O(\text{rerank_k} \cdot N)$ 复杂度, 是本实现在 GPU 上获得 200x 以上加速比的关键。

Algorithm 5 LSH - GPU: 设备端查询

Input: `dBase`, `dCodes`(库二值码), `dPlanes`, `dQ`(B 条查询), `rerank_k`, k

Output: B 条查询的 top- k 下标 `dIdx`

- 1: $\text{dProj} \leftarrow \text{cublasSgemm}(\text{dQ}_{B \times d} \cdot \text{dPlanes}_{K \times d}^\top)$ ▷ 投影 = GEMM
 - 2: $\text{dQcodes} \leftarrow \text{lsh_pack_kernel}(\text{dProj})$ ▷ 符号位打包
 - 3: $\text{dHam}[b, i] \leftarrow \text{lsh_hamming_kernel}(\text{dQcodes}, \text{dCodes})$ ▷ $B \times N$ 汉明距离
 - 4: **for** each query block b **do**
 - 5: 共享内存直方图: $\text{hist}[h] \leftarrow \text{汉明距离} = h \text{ 的向量数}$
 - 6: 最小半径 r : $\sum_{h=0}^r \text{hist}[h] \geq \text{rerank_k}$
 - 7: 收集 $\text{Ham} \leq r$ 的候选, 在共享内存中对候选做精确内积 $\rightarrow$ top- k
 - 8: **end for**
-

(三) 跨架构对比

引入 LSH 不仅可以补全 ANNS 算法中哈希范式这类方法, 同时还提供了一组完全相同的算法在两种架构上的对比, 从前文中可以看出, LSH 对于 GPU 的适配性远优于 CPU, 从结果上也能看出同一算法在不同架构上的差异:

表 1: LSH 跨架构对比 (均为 $K=256$, $\text{rerank_k}=500$, 8 线程)

架构	召回率	延迟 ($\mu\text{s}/q$)	加速比 vs Flat AVX2
CPU (AVX2+OpenMP)	0.9767	621	$2.5\times$
GPU (cuBLAS+CUDA)	0.9803	8.7	$179\times$

同样的 SimHash 算法, 在 CPU 上延迟高达 $621 \mu\text{s}/q$ (甚至慢于 IVF), 而在 GPU 上仅需 $8.7 \mu\text{s}/q$, 两者差距超过 70 倍。造成这一现象的根本原因在于投影的几何形态, CPU 需要逐条向量串行计算 K 个内积, 且汉明扫描仍是全库 $O(N)$; GPU 将投影转化为一次 GEMM、汉明转化为一次 element-wise kernel, 二者均为 GPU 最擅长的大块规整运算, 可以发挥其高度优化的矩阵运算优势。通过该对比也可看出, 算法自身是中性的, 但其在不同硬件上的效率往往具有很明显的差异, 这取决于其计算模式是否与硬件的并行原语相匹配。算法与架构的适配性也是后文会详细讨论的问题之一。

MINIBU

第二部分 综合并行实现

五、 算法-架构协同分流

(一) 设计动机与思路

回顾前五次实验中各自独立的 CPU/GPU 算法，以及新增的 LSH 算法在不同硬件上的差异，我们可以观察到一些重要的规律：

- **GPU 上, LSH 具有显著优势:** GPU-LSH 延迟仅为 $8.7 \mu\text{s}/q$ ($179\times$)，在保持高精度召回率 (0.98) 的同时，远超 GPU 精确暴力的 $25.1 \mu\text{s}/q$ ($62\times$)。这是因为 LSH 的汉明粗排在高成本 GEMM 精排之前先做了剪枝，在适配 GPU 运算的前提下进一步实现了算法策略上的优化。
- **CPU 上, IVF 优于 LSH:** CPU-IVF 在 $27 \mu\text{s}/q$ 时可达召回率 0.90，而 CPU-LSH 在同召回率下则需要 $300+ \mu\text{s}/q$ 。CPU 的标量/Popcount 很难在 10 万条的量级上发挥速度，而 IVF 的簇级剪枝则可以大幅缩小搜索空间，对于 CPU 的硬件结构来说更为适配。

通过这两条规律，我们也很容易想到，单一的策略往往难以在 CPU+GPU 的异构环境下发挥其最优的效果；而当回顾之前的实验时，MPI 多进程思想为我们的设计提供了参考思路：

是否可以将 CPU 与 GPU 当作两个独立的进程来对待，各自使用与自己架构最适配的算法进行运算，从而将各自的性能发挥到极致？

这样的异构分流机制也是本次实验提出的融合方法的核心基础。

(二) 融合策略一：异构协同分流

融合策略一的思路是在单台机器内实现 GPU 与 CPU 的并发协同。将查询集按比例 f 切分，前 $f \cdot M$ 条查询分配给 GPU 线程，使用 LSH 检索完成计算；余下的 $(1-f) \cdot M$ 条查询则分配给主线程，使用 OpenMP + SIMD(AVX2) 的 IVF 检索。GPU 线程与 CPU 主线程并发执行，通过调整合适的切分比例 f 使得 CPU 与 GPU 两侧同时结束，从而让现有资源的算力得到充分的发挥。

Algorithm 6 融合一：异构协同分流 (S5)

Input: base, query[M], GPU 占比 f , LSH 参数 $K, \text{rerank_k}$, IVF 参数 nprobe , 线程数

Output: 每条查询的 top- k

- 1: ▷ 准备 (计时区外)
 - 2: GPU: 上传 base 至 dBase, 建 LSH 索引 (planes $\rightarrow$ dCodes)
 - 3: CPU: 建 IVF 索引 (KMeans 聚类 + 内存重排)
 - 4: $g \leftarrow \lfloor f \cdot M \rfloor$
 - 5: ▷ 并发检索 (计时区)
 - 6: GPU 线程: `lsh_gpu_search(query[0 :  $g$ ], rerank_k)` ▷ 异步投影 + 汉明 + 精排
 - 7: CPU 主线程: `cpu_ivf_search_range(query[ $g$  :  $M$ ], nprobe)` ▷ OpenMP IVF
 - 8: 等待 GPU 线程 join; 归并两段结果
 - 9: **return** 完整 M 条查询的 top- k
-

GPU 分支保留 `gpu_algo` 开关，可以切换到 cuBLAS 精确检索的模式，以便后续对比精确算法与 LSH 算法的差异。

(三) 融合策略二：MPI + 异构分流

在融合策略一中，我们按设备并行，但实际上还可以在 CPU 内进行进一步的细分，即在 CPU 侧继续进行多进程并行。融合策略二将融合策略一的异构分流思想继续提升到 **MPI 进程级**。具体实现上，rank 0 作为 GPU 节点（运行 LSH / 精确），rank 1...R-1 作为 CPU 节点（运行 IVF+OpenMP）。各 rank 依照公式关于 f 计算出自己的 query 段，并发进行检索计算，最后再由 rank 0 通过 MPI_Gatherv 归并。此时最大延迟由各 rank 检索用时的最大值决定。

Algorithm 7 融合二：MPI + 异构协同分流 (S6)

Input: base, query[M], GPU 占比 f , 进程数 R

Output: 每条查询的 top- k

```

1: MPI_Init
2:                                     ▷ 计算切分（各 rank 用同一规则）
3:  $gpuN \leftarrow \lfloor f \cdot M \rfloor$ ; 其余  $M - gpuN$  均分给 rank 1..R-1
4: 各 rank 确认自己的  $[lo, hi]$ 
5:                                     ▷ 准备（计时区外）：rank0 建 LSH / 上载 dBase, 其余 rank 建 IVF
6: MPI_Barrier
7:                                     ▷ 并发检索（计时区）
8: rank 0 (GPU 节点) : lsh_gpu_search(query[ $lo : hi$ ])
9: rank 1..R-1 (CPU 节点) : cpu_ivf_search_range(query[ $lo : hi$ ])
10:  $t_{max} \leftarrow \text{MPI\_Reduce}(\text{MAX})$  各 rank 用时
11: MPI_Gatherv 归并结果到 rank 0
12: rank 0 计算综合召回率

```

对于该设计来说， f 的选取对算法的性能具有决定性的作用；而从硬件角度来看，最优的性能由 CPU/GPU 的算力比而决定。因此在本机的具体测试时，我们需要重点关注不同 f 带来的差异，探讨最优的分流比例；此外，对于不同的设备资源，最优的 f 往往也会有所差异，需要根据计算资源的具体情况进行多方面的考量。

六、实验结果与分析

(一) 实验环境与策略配置

本次实验均在本机环境完成：Intel Core i7 (x86, 8 物理核) + NVIDIA GeForce RTX 4060 Laptop GPU (8 GB)。数据集为 DEEP100K ($N=100000$, $d=96$)，使用 2000 条查询， $k=10$ 。统一采用 AVX2 暴力检索的延迟 ($1562.5 \mu\text{s}/q$) 作为所有策略加速比的基线。编译环境为 nvcc (OpenMP 经 `-Xcompiler /openmp` 生效)。本次实验实现了 S0-S6 共 7 个策略，各个策略的具体内容如下：

表 2: 策略对照表

编号	策略名称	说明
S0	Flat 暴力算法	单机 AVX2 精确暴力算法, 加速比基线
S1	CPU IVF / IVF-PQ	CPU AVX2-SIMD+OpenMP 多线程, IVF/IVF-PQ
S2	GPU cuBLAS	GPU cuBLAS SGEMM 精确暴力 + 设备 Top-k
S3	LSH - CPU	SimHash-LSH 纯 CPU 版 (AVX2+OpenMP)
S4	LSH - GPU	SimHash-LSH 纯 GPU 版 (cuBLAS+CUDA)
S5	异构协同分流	GPU-LSH CPU-IVF 异构协同
S6	MPI + 异构分流	MPI 进程级 GPU-LSH CPU-IVF 异构协同

(二) 数据总览

表 3 汇总了全部策略在代表性配置下的结果。

表 3: 各策略代表性结果总览

编号	策略 / 最优配置	召回率	$\mu s/q$	加速比	qps
S0	Flat 暴力算法 (AVX2)	1.0000	1562.5	1×	640
S1	CPU IVF nprobe=8, 8 线程	0.9024	26.9	58×	37k
S2	GPU cuBLAS exact	1.0000	25.1	62×	40k
S3	LSH - CPU K=256, rk=1000	0.9921	1116	1.4×	0.9k
S4	LSH - GPU K=256, rk=200	0.9355	7.3	213×	137k
S5	异构协同分流 LSH f=0.8	0.9757	8.3	188×	120k
S6	MPI + 异构分流 LSH f=0.75	0.9739	8.0	195×	125k

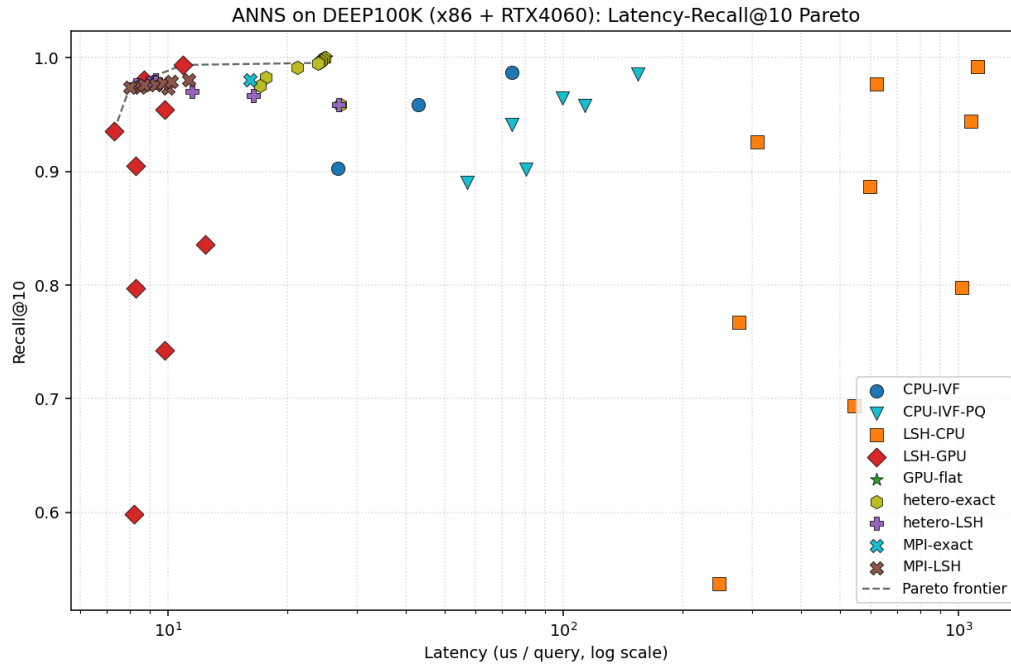


图 2: 各方法 Latency-Recall@k Pareto

总体来看, 各个策略差异显著, 其中以 LSH-GPU 和两类融合策略整体最优, 其余策略差距明显; LSH-CPU 在精度与速度上表现均为最差, 这也体现出 LSH 与 CPU 的严重适配性问题。

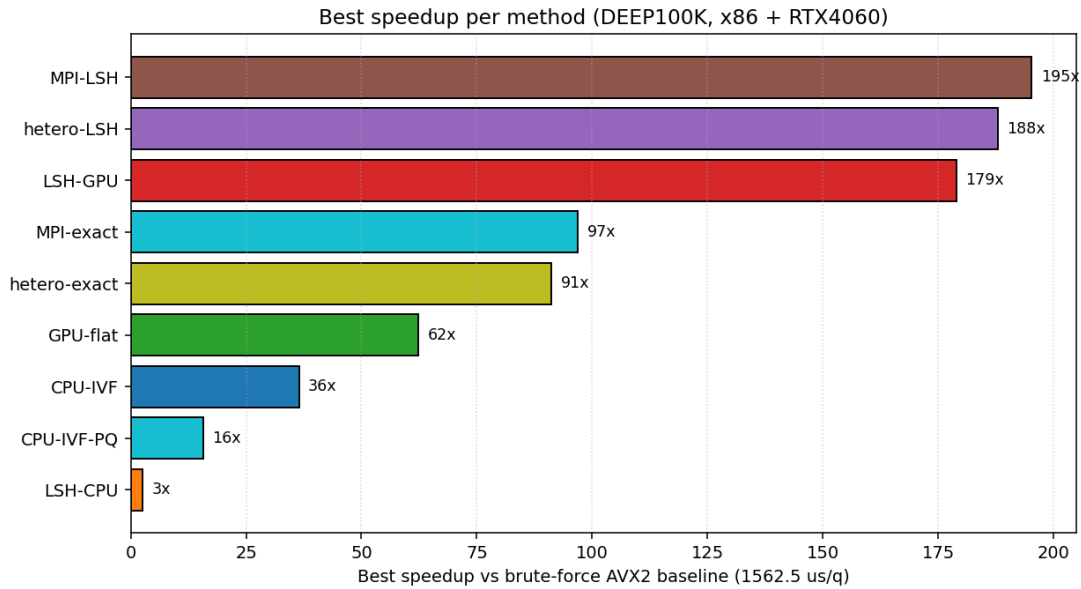


图 3: 各方法最优加速比对比 (Recall@k > 0.95)

从最优加速比来看，融合策略虽然相较单纯的 LSH-GPU 有所提升，但其优化效果并不算显著（179x/188x/195x）。其原因也与本机设备有关，实验所用 GPU（RTX4060）的性能相对于 CPU 具有明显的算力优势，且 LSH 在 GPU 上的优化效果也优于 IVF 在 CPU 上，因而在 CPU 上分流相对于纯 GPU 优势并不明显。尽管如此，实验结果的加速比提升也确实印证了这类异构融合方法的有效性。

接下来将对新增的几种策略进行详细的分析。

(三) LSH 参数权衡

S4 中 K （码长）和 rerank_k （候选数）的参数扫描（表 4）表明：随着 K 增大，汉明空间的保距性也在增强，召回率单调上升； rerank_k 增大意味着候选集的增大，也有益于召回提升，但其效果相较于 K 较为缓慢，且会造成延迟的提升。在追求速度的配置下（ $K=256$, $\text{rk}=200$ ），LSH-GPU 以 $7.3 \mu\text{s}/\text{q}$ 和 $213\times$ 加速比达到了本实验的最高加速比，但召回率仅 0.936；考虑均衡的场景， $K=256$, $\text{rk}=500$ 的配置下，可以达到 $8.7 \mu\text{s}/\text{q}$ 的延迟与 0.980 的较高召回。

表 4: S4 LSH-GPU: K 与 rerank_k 参数扫描（节选）

K	rerank_k	召回	$\mu\text{s}/\text{q}$	加速比
128	200	0.7970	8.3	188×
128	1000	0.9540	9.8	160×
256	200	0.9355	7.3	213×
256	500	0.9803	8.7	179×
256	1000	0.9935	10.9	144×

对于夹角为 θ 的归一化向量对， K bit 哈希码完全匹配的概率为 $P_{\text{match}} = (1 - \theta/\pi)^K$ 。当 K 从 64 增至 256，即使对 $\theta = \pi/12$ 这样的小夹角， P_{match} 也能从 $0.96^{64} \approx 0.07$ 优化至 $0.96^{256} \approx 2.9 \times 10^{-5}$ 。即 K 越大向量的方向一致性就越高，粗排阶段的召回质量也随之提升。

同时， K 的增大也几乎不会对于汉明距离的计算产生影响，两个 K 位码的比较仅需一次 XOR 和一次 popcount，x86 端对应 POPCNT 指令，CUDA 端对应 `__popc11()` 内置函数，均为单周期吞吐的硬件操作， $K = 256$ 与 $K = 64$ 在该环节的延迟差异在纳秒量级。此时查询延迟的真正瓶颈在于精排阶段，即对 rerank_k 个候选向量计算精确内积，而增大 K 则完全可以视为一个纯正向的参数选择。

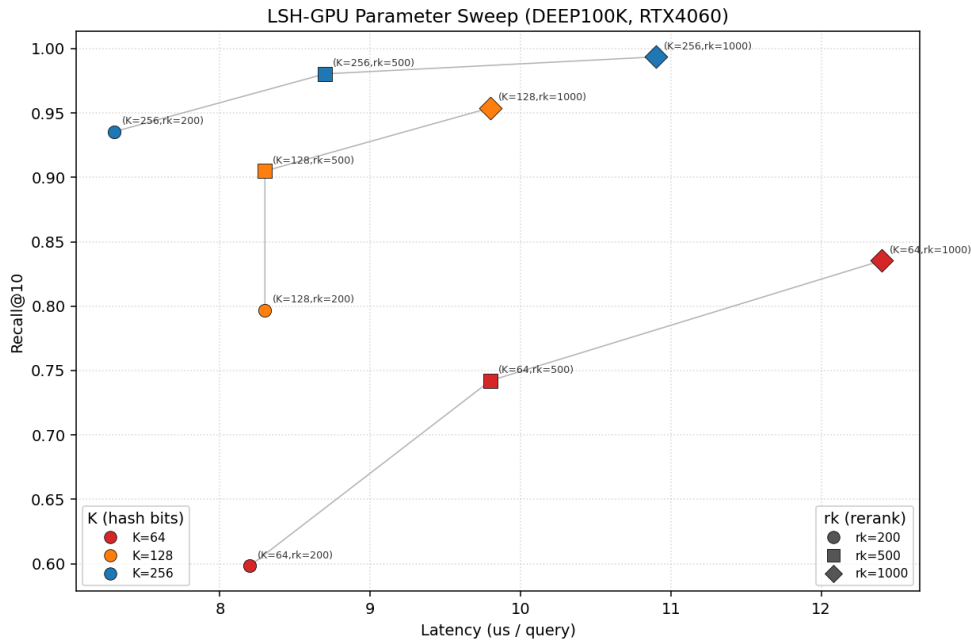


图 4: LSH-GPU 参数扫描

通过图像也可以看出, $K=256$ 具有明显的优势, 在延迟与召回率上均优于 $K=128/K=64$ 。在大部分场景中, 直接选用较高的 K ($K=256$) 即为最优的参数配置, 而后再根据具体需求调节 rerank_k , 在细节上对于精度与速度进行权衡即可。

(四) 算法与架构的适配与协同

S3 与 S4 提供了本报告中差异最为显著一组数据: 使用完全相同的 SimHash-LSH 算法, 在 CPU 上延迟为 GPU 上的 128 倍 ($621 / 8.7 \mu\text{s}/q$, $K=256/\text{rk}=500$)。更进一步地, 在 CPU 上考虑高召回的数据, LSH 甚至还可能出现慢于暴力基线的结果 ($1.4\times$)。

同样地, IVF 在 GPU 上的测试结果也不尽人意。对于同一对算法 (IVF / LSH), 在 CPU 和 GPU 上的优劣完全颠倒。这其实证明了算法的性能并不是算法自身的属性。一个算法是否高效, 实则取决于其计算模式是否与执行硬件的并行原语相匹配。CPU 擅长分支预测和低延迟串行逻辑, IVF 的簇级剪枝精准地缩减了搜索空间, 而 LSH 的 $O(N)$ 汉明扫描在 CPU 上则反而是无解的瓶颈; GPU 擅长海量线程下的规整矩阵计算, LSH 的“投影 = GEMM”精准地适配了这一擅长的模式, 运算下来反而比 IVF 要快的多。

S5 的 `gpu_algo` 开关提供了 GPU 侧的算法对照:

表 5: S5 异构协同分流: GPU-exact / GPU-LSH

GPU 分支算法	最优 f	召回率	qps	加速比
exact (cuBLAS 精确)	0.40	0.9752	58k	91 $\times$
LSH ($K=256$, $\text{rk}=500$)	0.80	0.9757	120k	188$\times$

将 GPU 分支的算法从精确 cuBLAS 替换为 LSH, 整个异构系统的吞吐从 58k qps 跃升至 120k qps, 仅凭算法的匹配就获得了 2.1 倍的吞吐提升, 而其召回率不变。这不仅验证了算法-架

构协同设计的原则，也从数据层面证明了两者的适配时带来的可观收益。

(五) 分流占比 f 的选取

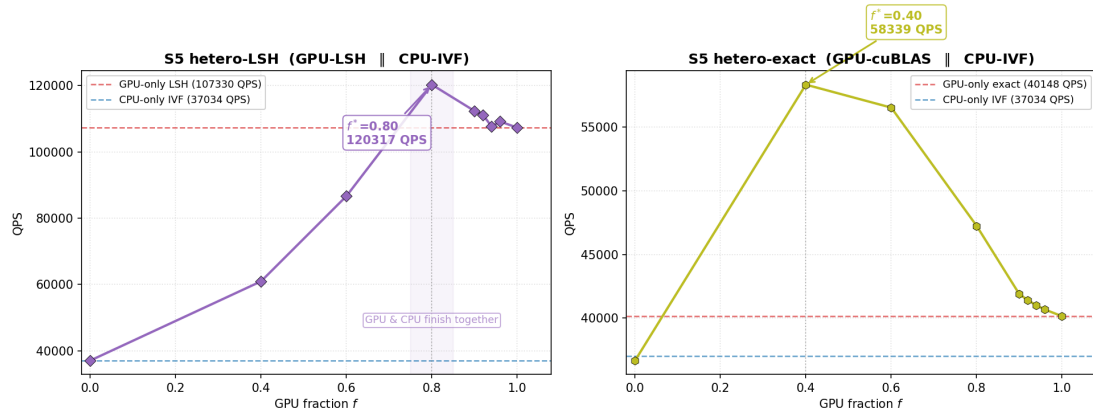


图 5: S5 hetero-LSH / hetero-exact 的 QPS 随 f 变化曲线

对比左右两图可以发现，无论 GPU 分支采用 LSH 还是精确 cuBLAS，QPS 随 f 的曲线均呈现明显两端低、中间高的凸形特征。hetero-LSH 在 $f = 0$ 时仅 37k QPS， $f = 1$ 时为 107k QPS，而 $f = 0.80$ 时达到峰值 120k QPS。hetero-exact 在 $f = 0$ 时约 37k QPS， $f = 1$ 时约 40k QPS，最优 $f = 0.40$ 处达到 58k QPS。凸形曲线的形状揭示了异构系统中两个硬件的相对算力关系。当 f 偏小时 GPU 过早完成分配给自己的少量查询后闲置等待，CPU 成为系统瓶颈。当 f 偏大时 CPU 同样提前完成而 GPU 成为瓶颈。仅在二者近乎同时完成的临界点附近，两路算力形成真正有效的并发叠加，吞吐达到最大值。图中 $f \in [0.75, 0.85]$ 的灰色阴影带标记了这一同时完成的大致区间，LSH 模式下的最优分流即在此区间内。

hetero-LSH 的最优 $f = 0.80$ 远高于 hetero-exact 的最优 $f = 0.40$ ，这一差异直接反映了 GPU 侧算法选择对异构系统负载分配的影响。LSH 模式下 GPU 的绝对吞吐为 107k QPS，是 exact 模式 40k QPS 的约 2.7 倍，因此 GPU 在单位时间内能够处理更多的查询，最优分流点自然向 GPU 侧倾斜，80% 的查询分配给 GPU，剩余 20% 分配给 CPU，两侧耗时方能匹配。而 exact 模式下 GPU 与 CPU 的吞吐处于同一数量级，最优分流点大约在 40% GPU 和 60% CPU，呈现更为均衡的分配。这从数据层面验证了前面的核心论点，即算法选择了硬件，而分流比例由算法与架构匹配后的实际算力比决定。

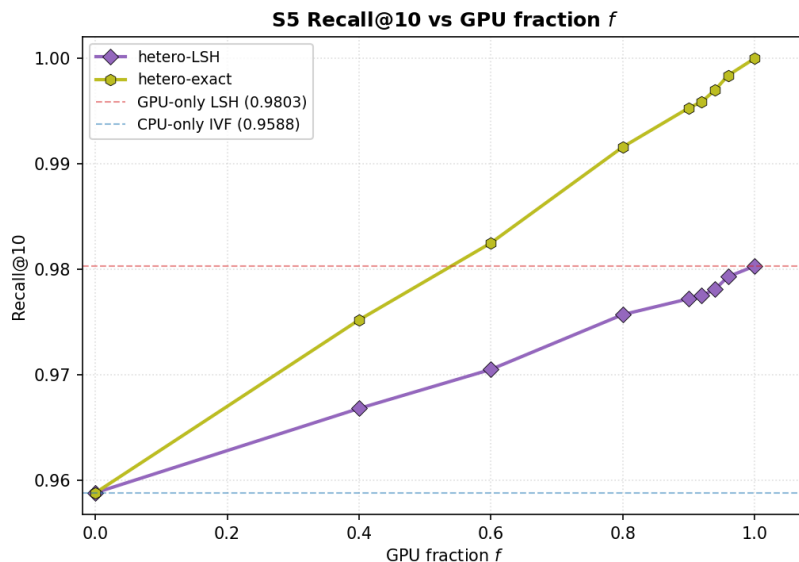
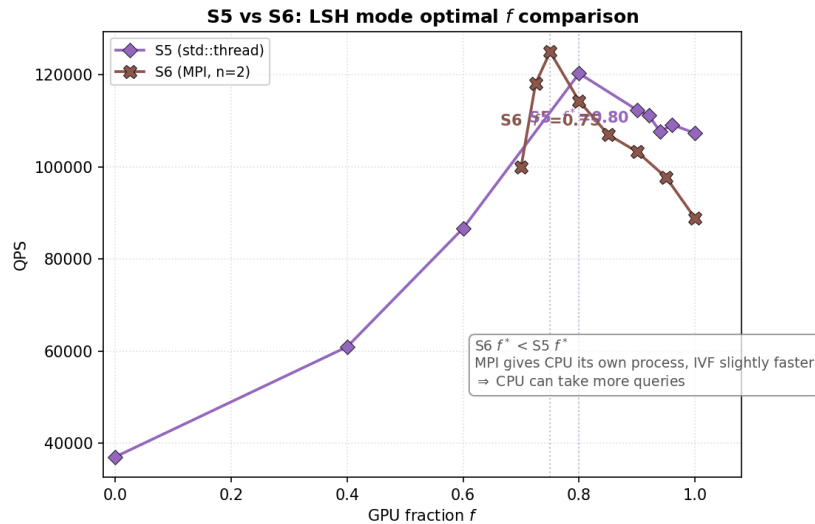


图 6: 召回率随 f 的变化

在召回率方面则差异较小，LSH 模式的召回率在全范围内变化幅度仅约 0.02 ($f = 0$: 0.959 / $f = 1$: 0.980)，exact 模式的变化幅度也仅约 0.04 (0.959 / 1.000)。这意味着可以在几乎不牺牲精度的前提下通过调节 f 来优化吞吐。召回率的轻微变化主要来源于 CPU-IVF 与 GPU-LSH 的精度差异，当 f 偏向 CPU 侧时更多查询由 IVF 处理，整体召回率向 IVF 的精度水平靠拢，反之亦然。

图 7: S5 / S6 最优 f 差异 (LSH)

观察 S5 与 S6 在 LSH 模式下的 QPS 曲线，可以看出两条曲线的整体形态高度一致，均在 $f \approx 0.75$ 到 0.80 附近达到峰值，这也表明了是分流逻辑本身而非并发机制决定了性能的上界。一个值得注意的差异是 S6 的最优 $f = 0.75$ 略低于 S5 的 $f = 0.80$ 。其原因在于 MPI 为 CPU 节点分配了独立的进程地址空间，CPU 侧 IVF 执行环境受到的 GPU 驱动线程和 CUDA 上下文等干扰略小于与 GPU 线程共享同一进程的 S5，因此 CPU 节点在单位时间内能处理略多的查询，最优平衡点相应向 CPU 侧偏移了约 5 个百分点。这一微小差异也从侧面印证了单机 MPI 异构分流中进程隔离带来的微弱收益，真实性能的提升仍需依赖跨节点的真实分布式部署。

综合以上分析，分流占比 f 是异构协同框架中唯一的自由参数，其最优值完全由参与协同的各硬件在其最优算法上的绝对吞吐之比决定。 f 的选取可以通过测取 GPU-only 与 CPU-only 的 QPS 来估算，记 GPU 独立吞吐为 Q_g 、CPU 独立吞吐为 Q_c ，则理论最优分流比为 $f^* = Q_g / (Q_g + Q_c)$ 。在本实验的 LSH 模式下 Q_g 为 107k QPS， Q_c 为 37k QPS，代入得理论值 $f^* = 0.74$ ，与实测最优值 0.80 接近但不完全一致。偏差主要源于 GPU 与 CPU 并发时的资源竞争，双方共享内存总线和末级缓存，实际有效吞吐均略低于独立运行时的测值，该效应在 GPU 吞吐远高于 CPU 时表现得更为明显，这也是这一思路后续扩展时的需要考虑的关键问题之一。

第三部分 总结与展望

七、 并行技术探讨

(一) 并行优化规律总结

回顾本学期从 SIMD 到 MPI 异构分流的完整实验历程, 尽管各实验面向的算法和硬件平台各不相同, 但其中反复出现的优化策略却呈现出高度一致的规律。将这些案例放在一起系统性地总结, 可以提炼出三条贯穿始终的并行优化原则。

第一条原则是粗排与精排的两阶段架构。这一模式在各类 ANNS 算法中以不同形式反复出现。PQ 用 ADC 查表粗排后用精确内积重排, IVF 用簇心距离粗排后在候选簇内精排, HNSW 用高层贪心导航粗排后在底层做集束搜索精排, LSH 用汉明距离粗排后精确内积精排。无论具体算法如何, 粗排阶段承担大幅剪枝的角色, 计算量大但每次操作廉价, 适合映射为规则的数据并行。精排阶段仅处理少量候选, 追求低延迟和精确性, 适合映射为任务级的动态负载均衡。理解这一分工, 也就理解了为什么 GPU-IVF 在簇数过多时性能崩溃, 不是算法有误, 而是本该属于粗排的簇心距离计算和本该属于精排的簇内 GEMM 被拆成了大量微小的 kernel 调用, 两阶段的差异被 grain size 的碎片化计算抹平, 从而无法发挥这一架构带来的收益。

第二条原则是任务粒度必须大于并行机制的开销阈值。并行并非总是优化, 每次线程创建、互斥锁争抢、kernel 启动或 MPI 消息传递都有固定的开销下限。IVF 粗排阶段的单线程设计就是这一原则的正面案例, $n_c = 256$ 个簇心的内积仅约 2.5×10^4 次乘加, 远小于线程创建开销, 对该部分不进行并行化处理反而是正确的选择。Pthread 在 $T = 8$ 时的扩展回退则是典型的反面案例, 单簇扫描仅耗时 5 至 10 μs , 而 `pthread_mutex_lock` 的开销与之处在同一数量级, 锁竞争完全吞噬了并行收益。GPU-IVF 在 $n_{\text{list}} = 256$ 时将矩阵乘拆分为大量微小 kernel 调用, kernel 启动延迟同样压倒了有效计算。这三个案例从不同角度指向同一个结论, 即并行化之前必须先判断被分配的单次任务计算量是否足够大, 否则并行不仅无法带来收益甚至可能反而产生负优化的结果。

第三条原则是同一优化思想在不同硬件上的形态由硬件的并行原语决定, 不能脱离硬件进行优化。SoA 布局在 CPU 上是为了匹配 SIMD 的连续加载指令和硬件预取器, 在 GPU 上则对应合并访存, 使 warp 内 32 个线程的全局内存访问合并为单次事务。同一个想法对应的是两种完全不同的实现路径。更深刻的例子则出现在粗排和精排的硬件迁移中, CPU 上 PQ 的 ADC 查表替代逐维浮点内积可带来数十倍加速, 而在 GPU 上 cuBLAS 的精确矩阵乘已经极快, 额外引入的 ADC 和逐簇 LUT 构建反而使整体延迟高于精确 IVF。在 CPU 上验证有效的优化迁移到 GPU 后结论完全相反, 这一现象反复地提醒我们, 优化的上限由硬件自身特性决定, 算法设计必须从目标平台的并行原语出发, 两者适配时才能发挥最优的效果。

以上三条原则, 两阶段架构的算法分工、任务粒度的工程约束、硬件原语的平台适配, 分别回答了 ANNS 并行优化中计算什么、如何分配、在哪运行三个基本问题。它们共同构成了后续异构协同设计的理论基础。

(二) 计算、访存与通信

将上述原则进一步抽象, 本学期的每一次优化本质上都是在计算、访存和通信三个维度之间做权衡。SIMD 通过单条指令处理多个数据元素来压缩有效指令数, 但其加速效率直接受限于内存对齐和带宽。OpenMP 将计算和访存分摊给多核, 但一旦线程数乘以单核带宽超过共享内存

总带宽，加速比便趋于饱和。MPI 将计算分摊到多个进程，但引入了节点间的集合通信开销，本实验中 $M = 2000$ 的查询规模下 Gatherv 仅传输约 80 KB，通信开销尚可忽略，但当查询规模扩展至百万级时结果集将膨胀至数十 MB，通信将成为新的瓶颈。GPU 以数千核心的海量并行带来数量级的算力提升，但同时引入了 Host-Device 数据传输和 kernel 启动的固定开销，只有规模足够大的规整矩阵运算才能充分摊薄这些开销。

单独启用某一层并行时，优化的方向是明确的，即找到当前层级的瓶颈维度，用该层级提供的并行机制去压缩它；但当多个层级同时启用时，各自的代价结构可能叠加而非抵消。本实验中 S6 的 MPI 异构分流就体现了这一问题，GPU kernel 启动开销、MPI 集合通信延迟和 CPU 侧 OpenMP 线程同步在单机上同时存在，三者各自的代价虽然都不高，但合在一起使 S6 的吞吐无法显著超越纯线程级的 S5。多层融合的根本困难不在于各层技术本身，而在于它们之间的代价重叠，工作重心也会随之从单个层级的优化转变为各项技术之间边界的处理。面对这种复杂的代价叠加，有时在细节上做特殊处理可以缓解，例如用非阻塞通信隐藏延迟；有时则需要在架构层面做出取舍，例如当 GPU 已经足够快时也可不必强行引入 MPI。

(三) 算法与架构的协同设计

而当考虑异构系统的设计时，我们面临的首要问题并非如何划分任务，而是为每种硬件选择什么样的合适的算法。算法的速度并不是内禀属性，而是其计算形态与硬件并行原语之间适配度的结果。SimHash-LSH 的跨架构对比为这一论断提供了最直接的数据支撑。完全相同的算法，在 CPU 上 $K = 256$ 、 $\text{rerank_k} = 500$ 时延迟为 $621 \mu\text{s}/q$ ，在 GPU 上仅为 $8.7 \mu\text{s}/q$ ，差距超过 70 倍。更关键的是算法优劣排序在两种架构上的颠倒，CPU 上 IVF 以 $42.9 \mu\text{s}/q$ 远优于 LSH 的 $621 \mu\text{s}/q$ ，而 GPU 上 LSH 以 $7.3 \mu\text{s}/q$ 超越所有方案，IVF 反而因逐簇 kernel 启动频繁而效率受限。同一对算法，换一个硬件平台，效果完全反转。

造成这种反转的根源在于两种硬件擅长的计算模式截然不同。CPU 以低延迟串行逻辑和分支预测见长，IVF 的簇级剪枝将搜索范围从全库 N 压缩至 $n\text{probe} \times N/n\text{list}$ ，在缩减后的候选集上做 SIMD 加速的精确内积扫描，整个流程高度适配 CPU 的 cache 层次与指令流水线。而 LSH 的汉明粗排再怎么优化也是 $O(N)$ 的全库扫描，在十万元素的数据规模上 popcount 硬件指令的速度优势被访存延迟完全覆盖。GPU 的局面则恰恰相反，LSH 的随机投影等价于矩阵乘法 $\mathbf{X} \cdot \mathbf{W}^T$ ，经 cuBLAS GEMM 一次调用即可完成全部投影计算，随后的汉明距离计算在设备端以合并访存方式执行，数千核心同时推进，单条查询的延迟被均摊到几乎可以忽略的水平。而 IVF 在 GPU 上按簇逐次调用 GEMM 退化为大量微小矩阵乘，kernel 启动开销成为绝对主导。

这一事实也直接决定了异构系统设计的方法与思路。传统做法先确定硬件分工再为各部分挑选算法，是自顶向下的架构驱动。更有效的方式是反向的基准驱动，先在不同硬件上独立运行所有候选算法，获得跨架构的性能矩阵，以每种硬件上表现最优的算法作为该硬件的专属执行体，再据此设计协同框架。GPU 分支运行 LSH 不是因为预先分配了 GPU 负责 LSH，而是因为基准测试表明 LSH 在 GPU 上的吞吐为 107k qps，远超包括精确暴力 40k qps 在内的所有 GPU 候选算法。CPU 分支运行 IVF 同理，在 CPU 上的吞吐 37k qps 和精度可控性均优于 LSH。算法选择了硬件，而非硬件选择算法。S5 GPU 上不同模式的对比实验也为这一原则提供了验证，固定异构框架不变，仅将 GPU 分支的算法从精确 cuBLAS 替换为 LSH，系统最优吞吐从 58k qps 跃升至 120k qps，提升 2.1 倍，而召回率不变。同一块 GPU 与同一套异构框架，唯一的变量是 GPU 上运行的算法，其结果差异显著。

(四) 异构融合的适用边界与分布式前景

算法与架构的匹配指明了每种硬件该跑什么算法，但对于解决异构融合本身是否总是必要，还需要更多方面的考量。本实验在 DEEP100K 数据集上恰好处于融合收益有限的区间，全精度基库约 38 MB，远小于 GPU 8 GB 显存，GPU 单卡即可独立完成全部检索。纯 GPU-LSH 单卡吞吐已达 107k qps，异构融合 S5 在此基础上提升至 120k qps，增益约 12%，具有正优化但效果并不显著。 f 扫描数据进一步验证了这一特征，仅在 $f \approx 0.80$ 附近，GPU 处理 80% 的查询耗时约 13 ms 而 CPU 以 8 线程 IVF 处理剩余 20% 也耗时约 13 ms，两路近乎同时完成，算力才形成有效的并发叠加。偏离此区间后，快的一方等待慢的一方，整体吞吐迅速退化。

异构融合产生显著收益需要两个条件同时满足。第一个条件是参与协同的各硬件在其最优算法上的绝对吞吐处于同一数量级。本实验中 GPU 精确暴力与 CPU-IVF 速度相近，融合带来 1.6 倍吞吐提升。而 LSH 模式下 GPU 吞吐是 CPU 的数倍，如果出现快的等待慢的，这也是一种特殊情况下的负载不均，融合增益难以进一步提升。第二个条件是数据规模大到使单一硬件无法独立承载。当数据量增长至亿级，全精度库约 38 GB 远超本机 GPU 8 GB 显存容量，CPU 侧的大容量内存将成为优势，索引存储与粗排剪枝必须驻留在 CPU 中，GPU 仅接收粗排筛选后的少量候选执行高速精排。在此场景下两种硬件各自承担对方无法替代的角色，异构融合才能真正带来决定性的加速收益。异构融合的价值不由架构的复杂度决定，而是由真实问题的瓶颈所在决定的。

S5 与 S6 的对比则揭示了另一个重要边界，体现出单机 MPI 的固有局限。两者实现了完全相同的异构数据并行逻辑，GPU 进程运行 LSH 而 CPU 进程运行 IVF，在最优配置下的吞吐几乎一致。这一结果并非偶然。MPI 的核心优势在于进程间地址空间隔离，天然适合跨节点分布式场景，但在单机上所有 MPI 进程仍然共享同一套物理资源，同一条内存总线、同一个末级缓存、同一块 GPU。进程隔离并未消除硬件层面的资源竞争，只是将竞争从线程调度层面转移到了操作系统进程调度层面。S6 的最优 $f = 0.75$ 略低于 S5 的 $f = 0.80$ ，正是因为 MPI 给 CPU 进程独立的地址空间使其 IVF 略快于线程共享模式，可以承担稍多的查询份额。但这一差异极其微小，远不足以让单机性能的上限得到突破性的提升。

真正分布式的 MPI 异构协同其实考虑的是一个完全不同的资源场景。在多节点配置下，GPU 服务器与 CPU 服务器各自独占内存总线和 PCIe 通道，节点间仅传输需要归并的 top- k 结果，不再存在单机场景下的带宽争抢。更进一步，还可将基向量库按数据并行的方式划分到各 CPU 节点，每个节点只持有全库的一个分片，本地 IVF 索引更小且构建更快，多节点联合覆盖的候选簇还可带来额外的召回率增益。本报告 S6 的代码在架构层面已为分布式部署做好了准备，各 rank 独立计算查询段范围并独立检索，通过 MPI 集合通信归并，这一流程无需修改即可在多机模式下运行。从单机验证走向多机部署需要解决的核心工程问题是各节点上索引数据的分发与同步，当前各 rank 各自从磁盘加载全量数据，在真正分布式场景下则需要改为各节点仅加载自己负责的分片。

回顾整个学期的实验历程，从 SIMD 指令级并行到分布式 GPU 异构协同，我们在 ANNS 这一个问题上走通了完整的多层次并行优化。这一路的经验最终指向同一个核心认识，即并行优化的本质不是技术的堆砌，而是在计算、访存与通信的复杂代价空间中，为每一层硬件选择最适合其并行原语的算法，并在代价重叠处做出合适的取舍。

八、 学期总结

(一) 实验回顾

本学期的五次实验报告与本次期末报告，从指令级并行到分布式异构并行，系统性地完成了 ANNS 问题并行优化的完整探索过程：

- **Lab1-2**: 理解了 CPU 微架构 (cache、分支预测、SIMD 寄存器) 对程序性能的底层影响，掌握了 SIMD 指令级并行的编程方法，并为后续实验构建了默认的 SIMD 基底。
- **Lab3**: 掌握了 OpenMP 多线程并行，通过线程内局部无锁堆、动态调度等技巧实现了簇级任务的负载均衡。
- **Lab4**: 掌握了 MPI 分布式并行，实践了数据并行与任务并行两种模式，探索了 MPI×OpenMP 混合编程。
- **Lab5**: 掌握了 GPU/CUDA 数据并行，理解了 SIMT 执行模型、Host-Device 数据搬运的代价、批量化与簇重叠率的概念。
- **期末报告**: 将上述四个层次的实验进行系统总结，引入 LSH 哈希范式，并通过算法与架构的协同提出了新的融合策略，进行了相应的实现与分析。

(二) 展望与心得

本次实验完成了从算法设计到系统融合的异构分流并行实验。沿着这一思路，还有许多值得进一步探索的方向：

- **OPQ 与 FastScan**: 旋转 PQ (OPQ) 通过最优旋转矩阵降低量化误差，FastScan 利用 SIMD 的查找表操作加速 ADC，这些也可以在 CPU IVF-PQ 端进一步缩小与纯 IVF 的差距。
- **GPU 图索引 (CAGRA)**: NVIDIA 在 cuVS 库中提出的 CAGRA 在 GPU 上实现了 HNSW 级召回-Speed Pareto，代表了 GPU ANNS 的新方向。
- **多 GPU 分布**: 将本报告的 MPI 融合二扩展至多 GPU 节点 (每节点一个 RTX4060)，每节点各自维护分片库，节点间通过 MPI 归并 top-k——这将是真正意义上的”分布式 GPU 异构协同”。
- **在线构建与流式更新**: 本报告所有实验均为离线构建 + 在线查询模型。实际向量数据库需要支持高效的插入/删除，如何在保持高速查询的前提下实现增量索引更新，也是工程化的关键挑战之一。

经过一个学期的《并行程序设计》的课程学习，从 CPU 微架构的指令流水线，到 GPU 数千核心的 SIMT，在学习与实验的过程中，让我对”并行计算”有了全新的立体认识。最深刻的体会在于，并行并不是把串行程序用多个线程重写一遍，而是从根本上重新思考问题与算法、硬件之间的关系。一个算法在 x86 上有效，不等于在 ARM 上有效；一个在 CPU 上最佳的优化策略，在 GPU 上可能毫无用处。真正的并行程序不是只会写 `#pragma omp parallel for` 就可以，而是能够理清问题模型背后复杂的层级关系，能够全面分析计算-访存-通信各个方面的代价，同时还需针对每一块硬件选择或设计最与之适配的算法。换言之，并行程序设计是一个不断发现问

题、解决问题的过程，所学到的各类并行技术只是一种工具，如何更好地利用这些并行技术，通过各种手段避免其自身的缺陷，使之可以更好地发挥其并行收益，才是这门课程为我带来的最大的收获。

学期的实验内容并不繁重，但每次实验都将我带入了一个真实而生动的问题场景，其中有过调试 Bug 的茫然，也有折腾半天测试出来负优化的挫败，而今回看这一学期的实验经历，也不由得感慨收获颇丰；编码能力的提升与分析思路的扩展，以及“并行”这一思想的深入理解，都让我有了深刻的体会。言至于此，本学期《并行程序设计》课程已圆满结束，最后衷心感谢王老师课堂上深入浅出的讲解与各位助教的耐心答疑，为我的课程学习带来了巨大的帮助。

NIJU

九、 附录：代码仓库链接

GitHub: <https://github.com/Iriasoul/Parallel-Programming/tree/main/lab06>

NIKU